GRAPHCORE RESEARCH

TECHNICAL REPORT

# Hardware-Aware FP4 FlashAttention-4

**REPORT FOCUS**

Pure-FP4 attention, fused probability quantization, and measured GB200/B300 speed–accuracy trade-offs.

**Robert Hu**

August 2026

**Abstract**

FlashAttention-4 (FA4) maps attention onto Blackwell's asynchronous matrix units, tensor memory (TMEM), TMA transfers, and online softmax. Although block-scaled FP4 matrix instructions target roughly four times BF16 throughput, the probability tile $P = \mathrm{softmax}(QK^\mathsf{T}/\sqrt{d})$ is created inside the kernel. It must be reduced, exponentiated, quantized, and published before FP4 PV can start. Existing implementations consequently find that FP4 QK with FP8 PV can be faster than FP4 QK and PV.

We retain HAO AI Lab's two-query TMEM pipeline and shorten its exposed P path. QK uses NVFP4, while P and V use MXFP4. A packed affine map sends normalized log scores directly toward E2M1 codes, the denominator is computed from the same represented probabilities consumed by PV, and native exponentials are used only where they improve the hardware schedule. For model layers with an extreme QK range, a sampled-row guard shares one exponent budget with the stored MX scale. Reordering the denominator multiplication prevents a valid minimum-scale block from flushing to zero, without adding an instruction.

On GB200, the fast policy is 2.125× faster than BF16 at B1/S8192/H64/D128. On B300 it reaches 3116 TFLOP/s at that shape and 3159 TFLOP/s on a wave-aligned case. HAO's NV/FP8 path remains the higher-fidelity reference; our result is a measured speed–accuracy frontier for full FP4 attention. The retained route remains finite in ViT, BERT, paired image reconstruction, and 20-step Wan replays, showing where operator error does and does not affect downstream behavior.

# 1 Introduction

Attention is usually written as

$$S = QK^\mathsf{T}/\sqrt{d}, \qquad P = \mathrm{softmax}(S), \qquad O = PV. \tag{1}$$

The fastest GPU implementation is determined not only by these equations but also by where each tile lives and when it becomes ready. FlashAttention avoids writing the quadratic $S$ and $P$ matrices to high-bandwidth memory [3]. Later versions improved work partitioning and adapted the pipeline to asynchronous matrix units [2, 8]. FlashAttention-4 (FA4) redesigns this schedule for Blackwell's tensor memory (TMEM), TMA transfers, and fully asynchronous tensor-core operations [10].

Blackwell also provides block-scaled FP4 matrix multiplication with a peak throughput target near four times BF16 on GB200. That ratio applies to QK and PV, but not to the work between them. Q, K, and V can be quantized before the attention kernel starts. P cannot: each online-softmax iteration must create, scale, pack, and publish P before the next matrix product can consume it. Making QK and PV shorter therefore makes this serial path more visible.

HAO AI Lab's exploratory FP4 FA4 implementation demonstrates the central counterexample: NVFP4 QK with FP8 PV is often faster than an all-NVFP4 route [4, 12]. Its two-query CTA and TMEM ownership model solve the large scheduling problem. We keep that structure and ask a narrower question: can the online probability representation be changed so that full FP4 PV becomes worthwhile?

Our retained format split is

$$\boxed{\text{NVFP4 } QK \quad + \quad \text{MXFP4 } P/V.} \tag{2}$$

NVFP4 keeps fine block-16 scales for signed Q and K. MXFP4 gives each block of 32 P and V values a power-of-two E8M0 scale. This wider range is useful for small probability blocks and lets scale selection fold into a log-domain code map.

This report makes four contributions:

1. It reconstructs HAO's two-query full-FP4 pipeline in ThunderKittens and states its TMEM ownership and N32-to-K64 publication rules explicitly.
2. It introduces an NV/MX probability path that maps log scores directly toward E2M1 codes with packed FMA, mixes native EX2 with arithmetic work, and normalizes by the probabilities that PV actually consumed.
3. It adds a low-cost range guard for extreme model logits. The final version enforces one exponent budget and uses an underflow-safe but algebraically equivalent denominator evaluation, fixing the observed 20-step Wan failure without a full row scan.
4. It evaluates latency and numerical error from matched records on GB200 and B300, compares against HAO and BF16, and adds ViT, BERT, ViT-MAE, and Wan model replays.

The scope is forward, noncausal D128 and D64 attention. Kernel timings include QK, online softmax, P quantization, PV, correction, and the output epilogue; they exclude dynamic Q/K/V quantization and optional K/V reordering. The paper first explains the inherited storage schedule, then the new probability path, followed by matched performance, accuracy, and remaining limits.

# 2 Why Full FP4 Is Difficult

## 2.1 P lies on the critical path

For one query row, exact attention is

$$O_i = \frac{\sum_j e^{z_{ij}} V_j}{\sum_j e^{z_{ij}}}, \qquad z_{ij} = Q_i K_j^{\mathsf{T}} / \sqrt{d}. \tag{3}$$

FlashAttention visits the keys in tiles. After a tile with maximum $\widetilde{m}_i$ arrives, it updates a running maximum $m_i$, denominator $\ell_i$, and output $o_i$:

$$m_i' = \max(m_i, \widetilde{m}_i), \tag{4}$$

$$\ell_i' = e^{m_i - m_i'} \ell_i + \sum_{j \in B} e^{z_{ij} - m_i'}, \tag{5}$$

$$o_i' = e^{m_i - m_i'} o_i + \sum_{j \in B} e^{z_{ij} - m_i'} V_j. \tag{6}$$

This avoids storing full score and probability matrices in HBM, but it also means P is not an input. For full FP4 attention, the first useful PV command waits for

$$S \to \text{maximum} \to \text{scale} \to \text{probability} \to \text{E2M1 pack} \to \text{publish}. \tag{7}$$

Work removed after publication may not change latency. Work removed before the first legal P tile can.

On data-center Blackwell GPUs, asynchronous `tcgen05` operations accumulate FP32 score and output tiles in TMEM. TMA supplies shared-memory operands, and specialized warps issue matrix operations, compute softmax, and correct the online output [7, 10]. Matrix throughput grew faster than score-load, exponential, and synchronization throughput. FP4 accelerates both matrix products while adding P conversion between them, so this imbalance is stronger than in BF16 FA4.

## 2.2 NVFP4 and MXFP4 solve different numerical problems

Both formats use signed E2M1 payload values

$$\mathcal{F}_{\mathrm{E2M1}} = \left\{0, \tfrac{1}{2}, 1, \tfrac{3}{2}, 2, 3, 4, 6\right\} \tag{8}$$

plus sign. Their scales differ:

**Table 1:** Block-scaled FP4 formats used in this work.

| Format | Block | Scale | Main strength | Retained use |
|---|---|---|---|---|
| NVFP4 | 16 | E4M3, optional tensor scale | fine local placement | signed Q and K |
| MXFP4 | 32 | E8M0 power of two | wide exponent range | nonnegative P and V |

For an NVFP4 P block with maximum $a_B$ and global encode factor $G$, the decoded block scale is approximately

$$\widehat{s}_{\mathrm{NV}} = \frac{Q_{\mathrm{E4M3}}(Ga_B/6)}{G}. \tag{9}$$

If the E4M3 scale rounds to zero, the entire block disappears. A row-level stabilizer fixes the range, but its scale and inverse correction add state to the online path. MXFP4 instead selects a nearby power of two,

$$\widehat{s}_{\mathrm{MX}} \in \left\{2^{\lfloor \log_2(a_B/6)\rfloor}, 2^{\lceil \log_2(a_B/6)\rceil}\right\}, \tag{10}$$

which folds naturally into a base-two score transform and matches one N32 producer fragment. The choice is a latency–range trade-off, not a claim that E8M0 is intrinsically more accurate than E4M3.

Table 2 isolates that distinction from kernel scheduling. It forms exact Gaussian softmax probabilities, quantizes matched N32 blocks, renormalizes the represented P, and multiplies by the same FP32 V. Stabilized NVFP4 is the high-fidelity control; unscaled NVFP4 exposes underflow.

**Table 2:** Probability-format range at D128. "Zero scales" is the fraction of blocks whose scale encodes as zero; "lost mass" is exact probability mass mapped to zero. This is a numerical diagnostic, not a kernel timing.

| $S$ | Format | Zero scales | Zero payload | Lost mass | $P$ rel.-$L_2$ | $PV$ cosine | $PV$ rel.-$L_2$ |
|---|---|---|---|---|---|---|---|
| 1024 | NVFP4 G=1 | 0.706299 | 0.831829 | 0.683773 | 1.715156 | 0.700819 | 1.685442 |
| 1024 | NVFP4 G=448 | 0.000000 | 0.155418 | 0.029314 | 0.114408 | 0.993854 | 0.113687 |
| 1024 | MXFP4 | 0.000000 | 0.188683 | 0.040178 | 0.146380 | 0.989846 | 0.144944 |
| 4096 | NVFP4 G=1 | 0.996063 | 0.999438 | 0.994332 | 13.815642 | 0.217328 | 13.876892 |
| 4096 | NVFP4 G=448 | 0.000000 | 0.156454 | 0.029760 | 0.114091 | 0.993747 | 0.114582 |
| 4096 | MXFP4 | 0.000000 | 0.189755 | 0.040494 | 0.147209 | 0.989338 | 0.148716 |
| 8192 | NVFP4 G=1 | 0.999634 | 0.999977 | 0.999284 | 12.467307 | 0.096774 | 12.389232 |
| 8192 | NVFP4 G=448 | 0.000000 | 0.160613 | 0.030691 | 0.114725 | 0.993841 | 0.113660 |
| 8192 | MXFP4 | 0.000000 | 0.194189 | 0.041630 | 0.147318 | 0.989660 | 0.146237 |

## 2.3 Relation to prior low-precision attention

HAO's FP4 FA4 repository is the structural starting point of this work [4]. It supports block-scaled FP4 QK with BF16 or FP8 PV and a stabilized full-NVFP4 path. Its implementation and the Attn-QAT study identify online P quantization and scale movement as the central cost [12].

SageAttention3 contributes Q/K smoothing, two-level P scaling, and reuse of the block maximum during online softmax [11]. Its RTX 5090 target is SM120, which does not expose the data-center `tcgen05` TMEM family used by SM100 and SM103 [7]. We therefore borrow its numerical lessons but use HAO and FA4 as the scheduling references. Our focus is forward kernel performance; we do not claim the training recovery shown by Attn-QAT.

# 3 Inherited HAO Pipeline

## 3.1 Two queries share one CTA

HAO's central contribution is a complete ownership plan for Blackwell [4]. For D128, one 16-warp CTA advances two independent M128 query tiles, called stage 0 and stage 1. A load warp keeps K and V ahead in a TMA-fed shared-memory ring. One warp issues ordered QK and PV operations. Two softmax warpgroups process the query stages in ping-pong fashion, while separate correction and epilogue roles update online state and store the result.

Small hardware barriers act as event counters, not block-wide meetings. They announce "score ready", "first P half ready", "tail P ready", and "output released". Algorithm 1 states the retained ownership protocol. Operations for $q = 0$ and $q = 1$ are interleaved whenever their dependencies allow; the order shown is per score bank, not a serialized whole-CTA loop.

**Algorithm 1** Two-query score-to-PV ownership protocol inherited from HAO

| | Owner | Event for query stage $q \in \{0, 1\}$ |
|---|---|---|
| 1 | Load warp | Prefetch the next K/V tile into the shared-memory ring. |
| 2 | MMA warp | Wait until score bank $S_q$ is free; issue QK into $S_q$ and publish *score ready*. |
| 3 | Softmax WG $q$ | Read N32 quarters Q0 and Q1, create packed P and scales in retired score addresses, then publish the first legal K64 operand. |
| 4 | MMA warp | Observe the first-half event and issue asynchronous $PV_{q,0}$ into permanent output bank $O_q$. |
| 5 | Softmax WG $q$ | Process Q2 and Q3 and publish the tail K64 operand. |
| 6 | MMA warp | Issue $PV_{q,1}$; meanwhile issue legal work for stage $1 - q$. |
| 7 | Correction WG | Update online state; the epilogue normalizes and stores when the key loop ends. |
| 8 | MMA warp | After tail PV has consumed the overlay, publish $S_q$ as free for the next QK tile. |

The two-query topology is essential at high head count: one CTA performs two query jobs while reusing the same K/V stream. A one-query topology creates twice as many independent CTAs and can add an extra GPU wave. A third query stage would provide more look-ahead, but the TMEM layout leaves no bank for it.

## 3.2 TMEM is full, so score storage must be recycled

SM100 and the tested SM103 path expose 512 logical TMEM columns. A D128 FP32 score tile or output accumulator uses 128 columns. Two score banks and two output banks therefore consume the entire allocation.

Each score bank is reused as

$$\text{QK score} \rightarrow \text{read N32 fragment} \rightarrow \text{P/scale overlay} \rightarrow \text{PV consume} \rightarrow \text{free}. \tag{11}$$

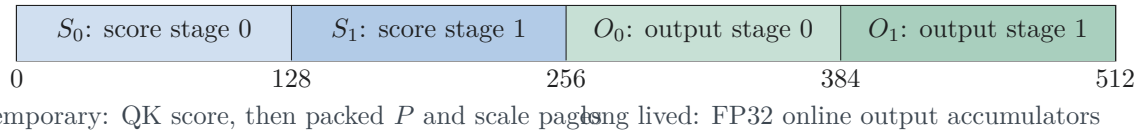


**Figure 1:** D128 TMEM layout inherited from HAO. P and its instruction-facing scales reuse score storage only after that score fragment is retired. There is no unowned 128-column bank.

Writing the next QK tile too early destroys P still owned by PV. Waiting too long leaves the matrix issuer idle. The scheduling problem is therefore an ownership handoff, not merely a shortage of barrier signals. A barrier can report that storage is free; it cannot create another destination.

### 3.3 Quartering creates an early publication point

QK still writes one M128×N128 FP32 score tile. "Quartering" means that a softmax warpgroup reads and transforms four N32 fragments,

$$S = [Q0 \mid Q1 \mid Q2 \mid Q3], \qquad Qk \in \mathbb{R}^{128\times 32}, \tag{12}$$

not that QK becomes four smaller matrix products. The score allocation remains 128 columns.

The available scaled-FP4 PV path consumes K64 rather than K32. Two adjacent quarters must therefore be complete before useful matrix work can issue:

$$(Q0, Q1) \to PV^{(0)}_{K64}, \qquad (Q2, Q3) \to PV^{(1)}_{K64}. \tag{13}$$

Quartering reduces the live register fragment and lets P overwrite retired score addresses. It does not reduce score-bank TMEM. Its performance value is the event after Q1: first-half PV can begin while Q2/Q3 and the other query stage provide independent work.

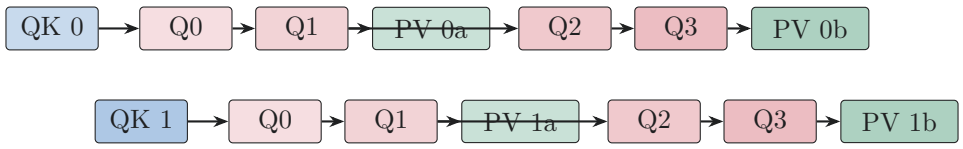


**Figure 2:** Dependency sketch for the two query stages. Box width is not measured time. Each first PV half waits for two N32 producer quarters; work from the other stage covers part of that delay.

## 4 NV/MX Probability Path

### 4.1 Design boundary

HAO solves the outer storage and scheduling problem; this work changes the path from a ready FP32 score fragment to a legal FP4 PV operand. The boundary is summarized below so that inherited machinery is not presented as a new contribution.

Q, K, and V are prequantized. QK uses NVFP4 with adjacent K64 Q/K scales folded offline by an MSE rule. For P/V, each N32 probability block uses one MXFP4 E8M0 scale. P payload and scale pages overwrite retired score addresses, exactly as required by the lifecycle in Algorithm 1.

**Table 3:** Inherited structure and changes made in this work.

| Component | Inherited from HAO | This work |
|---|---|---|
| CTA and TMEM | Two query stages, two score banks, two FP32 output banks, one ordered MMA issuer. | Retained. |
| P publication | N32 producer quarters, first-half and tail barriers, score/P overlay. | Retained; less work before each event. |
| Formats | NVFP4 QK and stabilized NVFP4 P/V. | NVFP4 QK, MXFP4 P/V. |
| P arithmetic | Stabilized exponential, P quantization, and scale publication. | Direct log-score-to-E2M1 map with selective EX2. |
| Normalization | Floating probability path. | Sum the represented P consumed by PV. |

## 4.2 Map scores to E2M1 codes directly

Let $m_i$ be a row reference, $s_B = 2^{e_B}$ the selected MX scale for one N32 block, and represent a probability as

$$\widetilde{p}_{ij} = s_B q_{ij}/6, \qquad q_{ij} \in \mathcal{F}_{\mathrm{E2M1}}. \tag{14}$$

The ideal converter input is

$$x_{ij} = (z_{ij} - m_i)\log_2 e - e_B + \log_2 6, \tag{15}$$

$$q_{ij} = Q_{\mathrm{E2M1}}(2^{x_{ij}}). \tag{16}$$

Only the code $q_{ij}$ reaches PV. Computing an accurate FP32 $2^x$ and then discarding almost all of its precision is unnecessary.

Positive E2M1 changes value at seven rounding boundaries,

$$\left\{\tfrac{1}{4}, \tfrac{3}{4}, \tfrac{5}{4}, \tfrac{7}{4}, \tfrac{5}{2}, \tfrac{7}{2}, 5\right\}. \tag{17}$$

We therefore fit an affine classifier in value space,

$$\widehat{u}(x) = \max(0, Ax + B), \qquad \widehat{q}(x) = Q_{\mathrm{E2M1}}(\widehat{u}(x)), \tag{18}$$

to maximize E2M1 code agreement rather than real-valued exponential accuracy. The score transform, $e_B$, and $\log_2 6$ term are folded into the packed-FMA coefficients. One `FFMA2` handles two scores, and packed `F2FP` emits their E2M1 payloads. A selected pair may instead use native EX2 when that improves the machine schedule.

**Algorithm 2** Direct MXFP4 probability production for one N32 score fragment

1. Load 32 scores per row from the completed TMEM score tile and reduce the block maximum.
2. Select power-of-two block scale $s_B = 2^{e_B}$ from that maximum and the row reference $m_i$.
3. For each packed score pair, form $x = (z - m_i)\log_2 e - e_B + \log_2 6$.
4. Evaluate either $\max(0, Ax + B)$ or selected native $2^x$, then use packed conversion to emit E2M1 codes $q$.
5. Pack four payload words and decode their small represented sum $c_B = \sum_{j \in B} q_{ij}$.
6. Accumulate denominator contribution $d_{iB} = s_B(c_B/6)$, using the same scale and codes as PV.
7. Overlay P payload and the swizzled scale page onto retired score addresses. Publish after two adjacent N32 fragments form one legal K64 operand.

The generic `fast` fit uses $A = 1.50, B = 1.20$. Wan activation replays select the equal-cost pair $A = 1.60, B = 0.95$. These constants move the seven decision boundaries,

$$x_j = (t_j - B)/A, \tag{19}$$

where $t_j$ is an E2M1 boundary. They are quantizer-calibration parameters, not changes to exact softmax. Layer-wise maps were tested but not retained: isolated substitutions on a BF16 teacher trajectory did not predict the composed all-FP4 trajectory (Appendix B.5).

Integer threshold trees, LUTs, and custom nibble packing generated more SASS than packed FMA followed by Blackwell's native converter. Quadratic and cubic fits improved real-function error but placed extra dependent FMA operations directly before publication.

## 4.3 Normalize the represented probability

For block $B$, the numerator and denominator actually consumed by the approximate operator are

$$\widetilde{N}_{iB} = \frac{s_B}{6} \sum_{j \in B} q_{ij} \widehat{V}_j, \tag{20}$$

$$\widetilde{L}_{iB} = \frac{s_B}{6} \sum_{j \in B} q_{ij}. \tag{21}$$

The output is

$$\widetilde{O}_i = \frac{\sum_B \widetilde{N}_{iB}}{\sum_B \widetilde{L}_{iB}}. \tag{22}$$

This avoids normalizing an E2M1 numerator with an unrelated approximate FP32 exponential sum. Four packed payload words are reduced with byte permutations, carry-free byte addition, and one `DP4A`. Streaming each word or keeping pairwise partials is algebraically equivalent but slower because it extends live ranges and changes instruction interleaving.

FA4 routes some exponential work from SFUs to ordinary arithmetic pipelines [10]. We apply the same balancing principle to the code map. GB200 D128 `fast` is all-affine; `accurate` uses native EX2 on roughly one quarter of pair positions. B300 has stronger SFUs, but broad EX2 routing still loses. Its retained policy uses native EX2 only in the quarter and shape regimes where the measured dependency schedule benefits.

## 4.4 Range guard for extreme model logits

The fast shiftless path is finite on the synthetic grid and most model layers, but a few late Wan layers produce BF16 logits above 500 and sometimes 1000. Scanning or reloading every score would remove the speed advantage. We instead route only those layers through Algorithm 3. Before launch, 128 globally distributed K/V rows are moved together into the first physical key tile. This permutation leaves exact attention unchanged.

**Algorithm 3** Sampled QK guard and underflow-safe represented denominator

1 In the first key tile, reduce all four N32 quarters and set sampled anchor $a_i = \max_{j \in \mathcal{A}} z_{ij}$, where $|\mathcal{A}| = 128$.
2 Choose compile-time margin $M$ and stored-scale shift $L$ with $M + L \le 126$; use row reference $m_i = a_i + M \ln 2$.
3 Run Algorithm 2. If its selected E8M0 code is $e_B$, publish translated code $e'_B = \max(1, e_B - L)$.
4 Let $s'_B = 2^{e'_B - 127}$ and $c_B = \sum q_{ij}$. Evaluate the represented denominator as $s'_B(c_B/6)$ in that order.
5 Never evaluate $(s'_B/6)c_B$: at $e'_B = 1$, the first product is subnormal and may flush a valid block to zero.

Wan1.3B uses $(M, L) = (110, 16)$ and Wan14B uses $(112, 14)$, so both consume the full 126-step budget. The bound is enforced at compile time and translated E8M0 scales are floored at code 1. The reordered evaluation uses the same two multiplies as the failing expression. On the original layer-39 failure, the sampled anchor already equalled the exact row maximum; the failure was solely the subnormal intermediate. This correction fixes it without a second scan, new barrier, or stable-softmax fallback.

### 4.5 Exposed policies

**Table 4:** Retained policies. Both use NVFP4 QK, MXFP4 P/V, K64 PV issue, and the two-query HAO layout.

| Policy | K/V stages | Anchor | Native EX2 | Denominator | Goal |
|---|---|---|---|---|---|
| `fast` | 12 | none by default | 0 on GB200 | producer | minimum latency |
| `accurate` | 13 | 32 fixed rows | about 25% | correction WG | higher fidelity |

The Wan bundle can add the 128-row routed guard to either base policy. Anchor permutations and folded Q/K scale preparation are outside the timed attention kernel and should be fused into an upstream layout or quantization step in a deployment.

## 5 Evaluation Method

### 5.1 Matched speed and error records

Each principal row stores latency and numerical error from one deterministic case. Intentionally incorrect ceilings and older runs with different seeds or timing windows are used only as diagnostics. For approximate output $\widehat{O}$ and BF16 reference $O$, we report

$$\cos(\widehat{O}, O) = \frac{\langle \widehat{O}, O \rangle}{\|\widehat{O}\|_2 \|O\|_2}, \tag{23}$$

$$\text{rel } L_2 = \frac{\|\widehat{O} - O\|_2}{\|O\|_2}, \tag{24}$$

$$\text{RMSE} = \sqrt{n^{-1} \sum_r (\widehat{O}_r - O_r)^2}. \tag{25}$$

Cosine can hide magnitude error and RMSE depends on output scale, so relative-$L_2$ is reported alongside both.

## 5.2 Operator benchmark suites

The primary GB200 suite reproduces the D128 shape grid in HAO's FP4 FA4 README:

$$\begin{aligned}(B, S, H) \in \{&(1, 256, 16), (1, 1024, 16), (4, 4096, 16),\\ &(1, 32768, 16), (4, 4096, 32), (1, 4096, 12), (1, 32768, 12),\\ &(1, 4096, 24), (1, 32768, 24)\}.\end{aligned} \tag{26}$$

Every case is noncausal D128, uses HAO's `create_nvfp4_attention_tensors` factory and seed 20260814, and is timed with 300 ms warmup and a 3000 ms median window. TK `fast`, TK `accurate`, native HAO NV/NV, and HAO CuTe-DSL BF16 receive the same prepared Q/K/V. The two TK binaries run in separate processes, but both use the controls from one canonical manifest:

$$\text{speedup}(k) = t_{\text{HAO BF16}}/t_k. \tag{27}$$

Throughput follows HAO's matrix-operation convention,

$$\text{ops} = BH\, 2S^2(D_{QK} + D_{VO}). \tag{28}$$

A saturation suite adds S4096/H64 and S8192/H64 plus local HAO and TK NV/FP8 controls. Its six-order harness removes short-case provider-order bias. A separate D64 matrix covers $S \in \{1024, 2048, 4096, 8192, 16384, 32768\}$ and $H \in \{12, 24, 32, 64\}$ on both GB200 and B300. Those 24 cross-generation cases share the input factory, seed, 100 ms warmup, and 1000 ms timing window.

B300 D128 rows are the best stable records from the final SM103 tuning campaign, not a same-binary hardware A/B. S4096 uses 300/3000 ms windows; S6144 and S32768 use repeated windows; S8192 and the wave-aligned S9472 case use five 2000-iteration windows and two correctness seeds. Published HAO B200 and GB300 values are labeled as cross-run context rather than local timings.

## 5.3 Timed scope and operand contract

Kernel time includes score loading, NVFP4 QK, online state, P approximation and packing, MX scale publication, MXFP4 PV, correction, and the output epilogue. It excludes dynamic Q/K/V quantization and optional K/V permutations, matching HAO's attention-kernel scope rather than full layer latency.

The fast path requires adjacent NVFP4 Q/K scale blocks folded for K64 access. The fold is chosen offline by reconstruction MSE. A production QKV projection should emit this layout directly. Pairing the binary with ordinary block-16 scales is an invalid operand contract even if its timing looks plausible.

## 5.4 Model and reconstruction replays

Six fixed replays replace every eligible attention call in ViT and BERT: ViT classification at S256, S1024, and S4096; BERT masked-language modeling at S256 and S512; and SST-2 classification at S256. All providers receive the same weights, examples, masks, prepared operands, and BF16 replay digest. We report attention-layer error, final logits or hidden-state error, prediction agreement, and task score. These are inference replays, not training-stability claims.

The ViT-MAE experiment replaces all 12 encoder self-attention layers for 100 COCO validation images with a fixed 75% patch mask [5, 6]. Its D64 model attention is padded to the existing

S256/H16/D128 specialization and padded keys are masked. Paired masked-patch PSNR, MSE, reconstruction cosine, and relative-$L_2$ are measured against the same BF16 decoder run.

Wan2.1 uses the native noncausal D128 topology at S7680: H12 across 30 layers for 1.3B and H40 across 40 layers for 14B [9]. Every video self-attention call is replaced; cross-attention and the rest of the model remain BF16. The fast route uses the global $A = 1.60, B = 0.95$ map. The sampled guard in Algorithm 3 is routed only to layers 27–29 in 1.3B and 33–34/38–39 in 14B. One-, four-, and twenty-step outputs are paired against HAO CuTe-DSL BF16 with identical prompt, seed, guidance, and initial latent. A second 14B prompt/seed checks the 20-step fix. Kernel timing uses independently warmed five-second windows and weights guarded and unguarded latencies by their layer counts.

Layer-wise affine calibration is kept as a negative control, not part of the reported route. Isolated substitutions were scored on a BF16 teacher trajectory, then rejected because the composed all-FP4 route changed sign between calibration and held-out prompts. The full control is in Appendix B.5.

### 5.5 Reproducibility

Generated tables are built from JSON manifests rather than hand-transcribed. Commands, policy manifests, compile-time constants, hardware metadata, job identifiers, and artifact locations are collected in Appendix E. Historical format paths and rejected timing experiments remain in the other appendices so they do not define the primary comparison.

## 6 Results

### 6.1 Operator performance and accuracy

Table 5 is the primary comparison. Panel A places the same finite NV/MX route on our GB200 and B300 systems and adds HAO's published GB300 NV/FP8 result where the shape matches. Panel B reproduces HAO's complete D128 grid locally on GB200 and reports both TK policies against one BF16 reference. Unbounded shiftless NV/NV timings are excluded even when a particular input happens to be finite.

Across the nine GB200 D128 rows, `fast` is 2.023× faster than HAO BF16 geometrically and peaks at 2998 TFLOP/s. `Accurate` reaches 1.669× and 2416 TFLOP/s. At S32768/H24, fast takes 4.400448 ms (2998 TFLOP/s), compared with HAO's published 2018 TFLOP/s on B200 and 2677 TFLOP/s on GB300 for NV/FP8.

HAO NV/FP8 remains more accurate: its mean cosine is 0.9899, versus 0.943789 for fast and 0.951669 for accurate. The corresponding TK mean relative-$L_2$ values are 0.336602 and 0.327225. HAO does not publish relative-$L_2$ for these rows. Appendix C measures the cost of matching its cosine with our exact NV/FP8 route; the local HAO NV/NV and full format matrix remain in Appendices A and D.

B300 improves D128 latency by 5.6–7.7% on the S4096–S8192 rows in Panel A. The standard S8192/H64 case reaches 3116 TFLOP/s, and wave-aligned S9472/H64 reaches 3159 TFLOP/s. The exception is S32768/H24: B300 reaches 2945 TFLOP/s versus 2998 on GB200. D64 shows a 4.7–6.0% B300 latency gain on the displayed rows. At HAO's published shapes, our B300 NV/MX reaches 2369 versus 2046 TFLOP/s at S4096/H24/D128, 2945 versus 2677 at S32768/H24/D128,

**Table 5:** Primary performance and accuracy comparison. Panel A contains local TK measurements; bold marks independently confirmed B300 results above 3 PFLOP/s. Panel B uses identical Q/K/V and BF16 outputs for both TK policies. Published HAO columns are cross-run context. "ms / TF" means milliseconds and TFLOP/s; TK error is cosine / relative-$L_2$ against BF16.

**A. Retained TK NV/MX across generations**

| Shape | TK GB200 ms / TF | TK B300 ms / TF | HAO GB300 NV/FP8 TF / cos. | B300 Δt | TK B300 cosine / rel.-$L_2$ |
|---|---|---|---|---|---|
| D128/H24/S4096 | 0.092160 / 2237 | 0.087008 / 2369 | 2046 / 0.9898 | -5.59% | 0.9438 / 0.3363 |
| D128/H64/S4096 | 0.202752 / 2711 | 0.189536 / 2901 | – | -6.52% | 0.9440 / 0.3355 |
| D128/H64/S6144 | 0.452848 / 2731 | 0.418107 / 2958 | – | -7.67% | 0.9432 / 0.3381 |
| D128/H64/S8192 | 0.758336 / 2900 | **0.705684 / 3116** | – | -6.94% | 0.944063–0.944113 / 0.334931–0.335152 |
| D128/H64/S9472 | – | **0.930724 / 3159** | – | – | 0.943960–0.944056 / 0.335198–0.335473 |
| D128/H24/S32768 | 4.400448 / 2998 | 4.480736 / 2945 | 2677 / 0.9899 | +1.82% | 0.9429 / 0.3389 |
| D64/H24/S4096 | 0.097888 / 1053 | 0.093248 / 1105 | – | -4.74% | 0.9556 / 0.2967 |
| D64/H64/S4096 | 0.221184 / 1243 | 0.207840 / 1323 | – | -6.03% | 0.9558 / 0.2960 |
| D64/H24/S32768 | 4.863296 / 1357 | 4.601808 / 1434 | 1203 / 0.9892 | -5.38% | 0.9561 / 0.2950 |
| D64/H64/S32768 | 12.865536 / 1367 | 12.223360 / 1439 | – | -4.99% | 0.9572 / 0.2915 |

**B. Complete published HAO D128 grid**

| Shape | TK NV/MX fast ms / TF | Error cos. / rel.-$L_2$ | TK NV/MX accurate ms / TF | Error cos. / rel.-$L_2$ | HAO B200 NV/FP8 TF | HAO GB300 NV/FP8 TF | HAO cosine NV/FP8 | HAO GB300 BF16 TF |
|---|---|---|---|---|---|---|---|---|
| B1/S256/H16/D128 | 0.010560 / 51 | 0.9445 / 0.3367 | 0.012288 / 44 | 0.9527 / 0.3242 | 39 | 11 | 0.9904 | 17 |
| B1/S1024/H16/D128 | 0.016704 / 514 | 0.9441 / 0.3357 | 0.018432 / 466 | 0.9519 / 0.3268 | 416 | 203 | 0.9901 | 289 |
| B1/S4096/H12/D128 | 0.064480 / 1599 | 0.9434 / 0.3376 | 0.077888 / 1323 | 0.9515 / 0.3287 | 1118 | 1458 | 0.9899 | 1017 |
| B1/S4096/H24/D128 | 0.092160 / 2237 | 0.9440 / 0.3358 | 0.110528 / 1865 | 0.9519 / 0.3266 | 1548 | 2046 | 0.9898 | 1322 |
| B4/S4096/H16/D128 | 0.200992 / 2735 | 0.9436 / 0.3370 | 0.248384 / 2213 | 0.9514 / 0.3279 | 1875 | 2502 | 0.9899 | 1508 |
| B4/S4096/H32/D128 | 0.391168 / 2811 | 0.9436 / 0.3367 | 0.489280 / 2247 | 0.9514 / 0.3277 | 1920 | 2540 | 0.9898 | 1475 |
| B1/S32768/H12/D128 | 2.279968 / 2893 | 0.9443 / 0.3348 | 2.867520 / 2301 | 0.9518 / 0.3262 | 1913 | 2582 | 0.9896 | 1584 |
| B1/S32768/H16/D128 | 2.964512 / 2967 | 0.9429 / 0.3387 | 3.726912 / 2360 | 0.9508 / 0.3295 | 2016 | 2666 | 0.9897 | 1585 |
| B1/S32768/H24/D128 | 4.400448 / 2998 | 0.9438 / 0.3364 | 5.461024 / 2416 | 0.9516 / 0.3275 | 2018 | 2677 | 0.9899 | 1533 |

and 1434 versus 1203 at S32768/H24/D64. These are different implementations and harnesses, so they establish scale rather than causal timing differences.

## 6.2 Speed–accuracy behavior across shapes

Figure 3 shows the independent B1/S4096/H24/D128 suite, including optimized local FP8-PV controls. Its six-order reference harness produces slightly different timings from Table 5, but all points share one BF16 value.

FP8 PV avoids the E2M1 payload and block-scale page required by full FP4 and is more accurate. The local TK NV/FP8 control reaches only 1.118× BF16 in this suite, whereas NV/MX fast reaches 1.783×; HAO's stronger published NV/FP8 result is therefore included in the primary table rather than inferred from this local control.

Figure 4 adds H64 cases. At S4096/H64, fast takes 0.202752 ms versus 0.370688 ms for BF16 (1.828×). At S8192/H64 it takes 0.758336 ms versus 1.611488 ms (2.125×). The two-query CTA is important here: the earlier one-query topology created twice as many jobs and could require an additional GPU wave at high head count.

## 6.3 ViT and BERT replays

Table 6 joins physical attention speed, mean layer error, and final task behavior. ViT uses 1,000 examples at S256 and 200 at S1024/S4096. BERT MLM uses 800 S256 blocks and 200 S512 blocks; SST-2 uses all 872 validation examples.

Both NV/MX policies remain finite. On ViT S4096, fast matches BF16 top-1 accuracy (88.5%) with 95.5% prediction agreement; accurate reaches 89.0% and 98.5%, while native HAO NV/NV

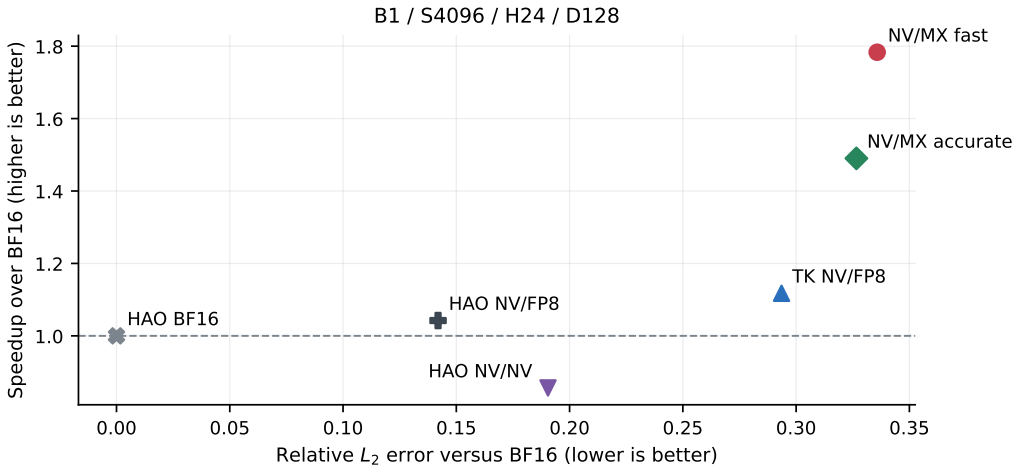


**Figure 3:** Speed–error plane for the headline shape. Up and left are better. The fastest full-FP4 points accept more operator error than HAO NV/NV or FP8 PV.

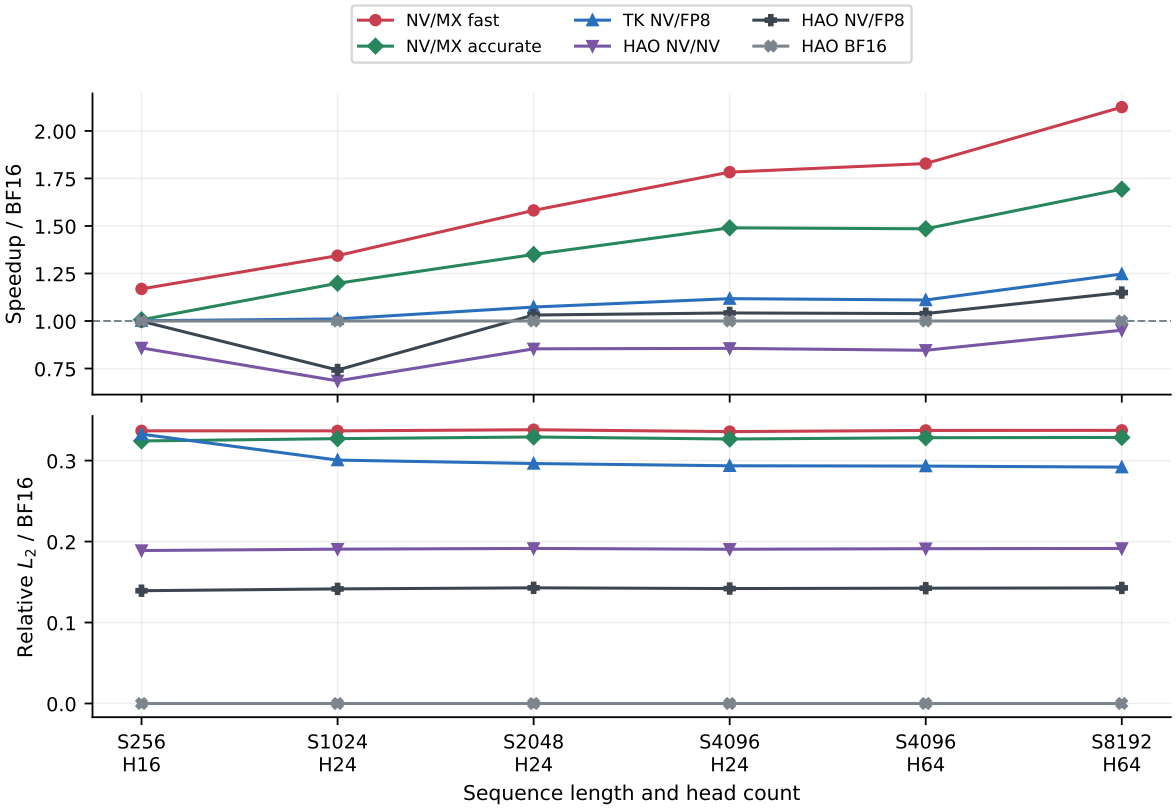


**Figure 4:** BF16 speedup and relative-$L_2$ from matched records across startup, transition, and saturated shapes.

**Table 6:** Downstream replay trade-off. Task score is provider / BF16 accuracy. Final and layer cells are cosine / relative-$L_2$ against BF16. Speedup belongs to the physical attention shape, not the complete model. HAO NV/NV is the identical-input control because no NV/FP8 task replay is published.

| Task | Provider | Speedup | Task score / BF16 (%) | Agree. (%) | Final cos. / rel.-$L_2$ | Layer cos. / rel.-$L_2$ | ΔMLM loss |
|---|---|---|---|---|---|---|---|
| ViT S256 | TK NV/MX `fast` | 1.169× | 98.80 / 98.90 | 99.30 | 0.9987 / 0.0514 | 0.9885 / 0.1506 | – |
| ViT S256 | TK NV/MX `accurate` | 1.007× | 98.70 / 98.90 | 99.60 | 0.9990 / 0.0452 | 0.9902 / 0.1401 | – |
| ViT S256 | HAO NV/NV | 0.858× | 98.70 / 98.90 | 99.80 | 0.9997 / 0.0238 | 0.9961 / 0.0872 | – |
| ViT S1024 | TK NV/MX `fast` | 1.344× | 95.00 / 96.50 | 97.50 | 0.9971 / 0.0768 | 0.9938 / 0.1093 | – |
| ViT S1024 | TK NV/MX `accurate` | 1.198× | 97.00 / 96.50 | 99.50 | 0.9984 / 0.0564 | 0.9949 / 0.1001 | – |
| ViT S1024 | HAO NV/NV | 0.685× | 97.00 / 96.50 | 99.50 | 0.9995 / 0.0324 | 0.9978 / 0.0652 | – |
| ViT S4096 | TK NV/MX `fast` | 1.783× | 88.50 / 88.50 | 95.50 | 0.9887 / 0.1509 | 0.9961 / 0.0886 | – |
| ViT S4096 | TK NV/MX `accurate` | 1.490× | 89.00 / 88.50 | 98.50 | 0.9989 / 0.0479 | 0.9971 / 0.0776 | – |
| ViT S4096 | HAO NV/NV | 0.856× | 88.00 / 88.50 | 98.00 | 0.9992 / 0.0396 | 0.9986 / 0.0517 | – |
| BERT SST-2 S256 | TK NV/MX `fast` | 1.169× | 92.32 / 92.43 | 98.74 | 0.9963 / 0.0870 | 0.9360 / 0.3547 | – |
| BERT SST-2 S256 | TK NV/MX `accurate` | 1.007× | 92.09 / 92.43 | 98.74 | 0.9967 / 0.0815 | 0.9507 / 0.3045 | – |
| BERT SST-2 S256 | HAO NV/NV | 0.858× | 92.20 / 92.43 | 99.77 | 0.9991 / 0.0431 | 0.9789 / 0.1890 | – |
| BERT MLM S256 | TK NV/MX `fast` | 1.169× | 61.26 / 61.74 | 90.61 | 0.9968 / 0.0796 | 0.9740 / 0.2302 | +0.035 |
| BERT MLM S256 | TK NV/MX `accurate` | 1.007× | 61.36 / 61.74 | 91.11 | 0.9971 / 0.0758 | 0.9768 / 0.2170 | +0.028 |
| BERT MLM S256 | HAO NV/NV | 0.858× | 61.60 / 61.74 | 93.06 | 0.9983 / 0.0577 | 0.9864 / 0.1626 | +0.016 |
| BERT MLM S512 | TK NV/MX `fast` | 1.344× | 61.10 / 61.79 | 90.17 | 0.9969 / 0.0782 | 0.9763 / 0.2176 | +0.037 |
| BERT MLM S512 | TK NV/MX `accurate` | 1.198× | 61.36 / 61.79 | 90.83 | 0.9971 / 0.0761 | 0.9779 / 0.2104 | +0.040 |
| BERT MLM S512 | HAO NV/NV | 0.685× | 61.28 / 61.79 | 92.30 | 0.9982 / 0.0604 | 0.9861 / 0.1636 | +0.028 |

reaches 88.0% and 98.0%. Fast improves from roughly 0.944 cosine in the Gaussian operator test to 0.9961 mean layer cosine on this model input. Residual paths, normalization, later mixing, and decision margins can attenuate operator error, so standalone relative-$L_2$ is informative but is not itself a task loss.

Across 2272 classification examples, fast changes 32 predictions, with 31 in the lowest quartile of BF16 top-two logit margins. All changes from accurate and HAO NV/NV also lie in that quartile. This supports a margin mechanism, but the suite is too small to establish universal inference or training safety.

## 6.4 Wan video diffusion

Wan is a larger, native-shape test: all video self-attention calls run at S7680/D128, while text cross-attention and the rest of the model stay BF16. Table 7 compares every route with paired HAO CuTe-DSL BF16 outputs and warmed kernel timing.

Fast is 1.75× faster than CuTe BF16 at 1.3B and 2.09× at 14B; accurate reaches 1.47× and 1.75×. HAO NV/FP8 is at parity for H12 and 1.18× faster for H40. At 14B/four steps, fast reaches 0.9338 / 0.3589, close to HAO NV/NV at 0.9327 / 0.3640. At twenty steps it reaches 0.8496 / 0.5337, versus 0.9036 / 0.4435 for HAO NV/NV; a held-out prompt reaches 0.8235 / 0.5813. The route is finite but accumulates more drift.

The original layer-39 failure was not an anchor miss: the sampled and exact row maxima were equal. The expression $(s/6)\sum_i c_i$ formed a subnormal intermediate at E8M0 code 1 and flushed to zero. Algorithm 3 reassociates it as $s(\sum_i c_i/6)$. Both 14B twenty-step runs now complete all 1,600 attention calls without a scan or stable-softmax fallback.

Guarded layers are 21–23% slower individually, but they are only 3/30 layers in 1.3B and 4/40 in 14B. Layer-weighted times are therefore 0.165309 ms and 0.424995 ms, 2.2–2.3% above the unguarded bases. The historical layer-wise affine map is omitted because its small gain reversed between prompts. These drop-in BF16-checkpoint results do not reproduce Attn-QAT's trained

**Table 7:** Wan2.1 quality and warmed GB200 kernel speed at S7680/D128. Speedup is relative to HAO CuTe-DSL BF16. Quality is cosine / relative-$L_2$ of the final latent against the paired BF16 run.

| Model | Method | Time (ms) | Speedup | 1 step | 4 steps | 20 steps |
|---|---|---|---|---|---|---|
| Wan2.1-1.3B | HAO CuTe BF16 | 0.2888 | 1.00× | 1.0000 / 0.0000 | 1.0000 / 0.0000 | 1.0000 / 0.0000 |
| Wan2.1-1.3B | TK NV/MX fast | 0.1653 | 1.75× | 0.9870 / 0.1803 | 0.9671 / 0.2611 | 0.9136 / 0.4163 |
| Wan2.1-1.3B | TK NV/MX accurate | 0.1961 | 1.47× | 0.9862 / 0.1721 | 0.9654 / 0.2637 | 0.9060 / 0.4323 |
| Wan2.1-1.3B | HAO NV/NV | 0.3466 | 0.83× | 0.9878 / 0.1751 | 0.9711 / 0.2405 | 0.9389 / 0.3463 |
| Wan2.1-1.3B | HAO NV/FP8 | 0.2888 | 1.00× | 0.9936 / 0.1184 | 0.9767 / 0.2158 | 0.9530 / 0.3066 |
| Wan2.1-14B | HAO CuTe BF16 | 0.8882 | 1.00× | 1.0000 / 0.0000 | 1.0000 / 0.0000 | 1.0000 / 0.0000 |
| Wan2.1-14B | TK NV/MX fast | 0.4250 | 2.09× | 0.9938 / 0.1179 | 0.9338 / 0.3589 | 0.8496 / 0.5337 |
| Wan2.1-14B | TK NV/MX accurate | 0.5072 | 1.75× | 0.9879 / 0.1563 | 0.9243 / 0.3946 | 0.8447 / 0.5500 |
| Wan2.1-14B | HAO NV/NV | 0.9073 | 0.98× | 0.9908 / 0.1358 | 0.9327 / 0.3640 | 0.9036 / 0.4435 |
| Wan2.1-14B | HAO NV/FP8 | 0.7516 | 1.18× | 0.9893 / 0.1464 | 0.9229 / 0.3877 | 0.8398 / 0.5669 |

quality; that requires its QAT checkpoint or training recipe [12].

### 6.5 Paired image reconstruction

Table 8 replaces all 12 ViT-MAE encoder attention layers for the same 100 images and masks. The S256 speedup is a physical kernel measurement, not end-to-end MAE speed.

**Table 8:** Paired ViT-MAE reconstruction. PSNR Δ is FP4 minus BF16 with a paired 95% interval. Reconstruction and layer cells are cosine / relative-$L_2$ against BF16.

| Provider | S256 speedup | PSNR (dB) | PSNR Δ (dB) | MSE Δ (%) | Reconstruction | Mean layer |
|---|---|---|---|---|---|---|
| BF16 | 1.000× | 23.917 | – | +0.00 | 1.00000 / 0.0000 | 1.0000 / 0.0000 |
| HAO NV/FP8 | 0.999× | 23.910 | -0.007 ± 0.012 | +0.16 | 0.99995 / 0.0089 | 0.9938 / 0.1046 |
| HAO NV/NV | 0.858× | 23.906 | -0.010 ± 0.015 | +0.07 | 0.99992 / 0.0113 | 0.9868 / 0.1589 |
| TK NV/MX `accurate` | 1.007× | 23.893 | -0.024 ± 0.021 | +0.25 | 0.99978 / 0.0184 | 0.9676 / 0.2616 |
| TK NV/MX `fast` | 1.169× | 23.896 | -0.020 ± 0.026 | +0.29 | 0.99973 / 0.0203 | 0.9600 / 0.2928 |

HAO NV/FP8 is closest to BF16 at $-0.007 \pm 0.012$ dB. HAO NV/NV, accurate, and fast reach $-0.010 \pm 0.015$, $-0.024 \pm 0.021$, and $-0.020 \pm 0.026$ dB. Fast has the largest reconstruction displacement, but its output remains 0.99973 cosine and 2.03% relative-$L_2$ from BF16 after all twelve layers. Figure 5 makes the residual visible only after an 8× amplification.

The aggressive shiftless TK NV/NV control fails on the first sample of every task, whereas HAO's stabilized NV/NV remains finite. Appendix A.7 shows that the distinction is stabilization, not the use of NVFP4 P itself.

## 7 Bottleneck Analysis

### 7.1 What still delays the first PV command

After QK completes, the first K64 PV half still depends on two N32 score fragments. Each fragment must be loaded from TMEM, reduced, assigned an MX scale, mapped and packed to E2M1, included

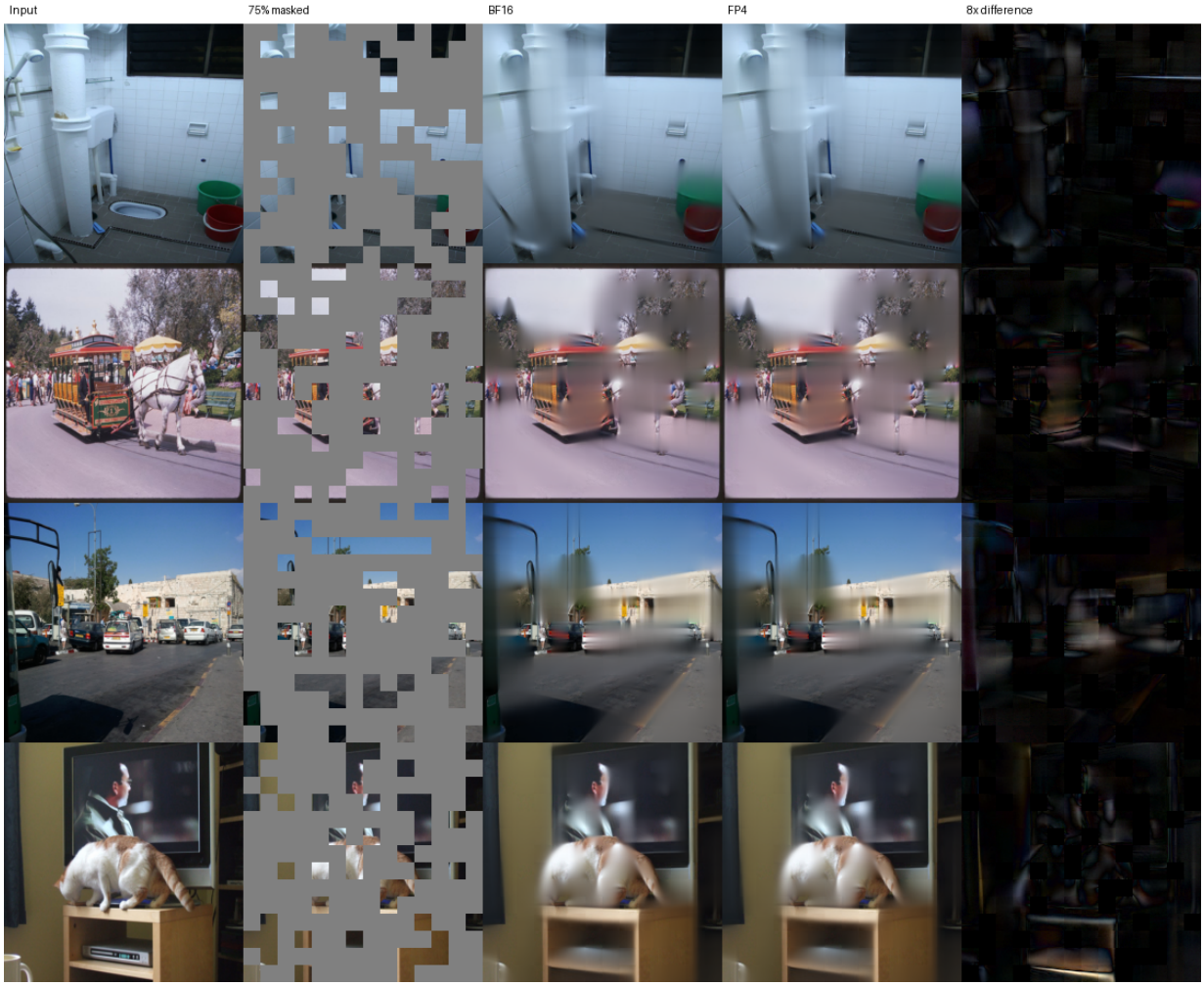


**Figure 5:** Four paired ViT-MAE examples. The right column shows $8|I_{\mathrm{FP4}} - I_{\mathrm{BF16}}|$; only encoder attention differs between the BF16 and fast runs.

in the represented denominator, written into the score overlay, and published. Q0 alone is not useful; Q1 must reach the same point before the matrix issuer has a legal operand. The other query stage and independent arithmetic hide part, but not all, of this chain.

At B1/S4096/H24/D128, a clean tuning record measured 0.092448 ms and repeated at 0.092512 ms. Three intentionally incorrect kernels bound the local headroom: The fixed-P experiment

**Table 9:** Speed-of-light diagnostics for the retained GB200 schedule. These rows do not compute valid attention.

| Diagnostic | Time (ms) | Gap from record |
|---|---|---|
| score-pack ceiling | 0.091168 | 1.280 $\mu$s (1.38%) |
| row max retained, raw score pack | 0.090112 | 2.336 $\mu$s (2.53%) |
| fixed P | 0.087616 | 4.832 $\mu$s (5.23%) |

removes nearly all P construction yet saves only 5.23%. Early full-FP4 kernels were dominated by P quantization; the retained NV/MX code path has removed most of that exposed cost. The remaining distance from four-times BF16 also contains TMEM score loads, K64 publication granularity, shared-memory delivery, two FP32 output accumulators, online correction, and the epilogue. Another local polynomial fit cannot remove those terms.

## 7.2 Instruction count is not the schedule

A matched fixed-P diagnostic cut dynamic instructions from about 54.6 million to 11.9 million and raised tensor-pipe activity from 18.8% to 26.6%. Other experiments removed arithmetic but became slower. Independent instructions had occupied scheduler slots while a TMEM dependency or barrier event was pending; removing them exposed the wait without advancing publication.

The represented-denominator decoder is a small example: All three are mathematically equivalent.

**Table 10:** Equivalent denominator reductions under the same schedule.

| Decoder | Time (ms) | Change |
|---|---|---|
| aggregate, retained | 0.092512 | baseline |
| pairwise partials | 0.094208 | +1.83% |
| stream after each word | 0.094464 | +2.11% |

The alternatives extend live ranges and alter compiler interleaving. This also explains why an integer threshold classifier lost to packed FMA plus native conversion despite looking simpler algebraically.

## 7.3 Why B300 is not uniformly 1.5 times faster

Blackwell Ultra raises dense NVFP4 throughput by 1.5×, doubles key SFU rates, and exposes fused TMEM load/reduction [1, 7, 10]. Those changes do not scale the whole attention loop. Every CTA still occupies all 512 TMEM columns, so residency remains one CTA per SM. Faster QK/PV makes score reduction, scale publication, online state, and the epilogue a larger fraction of time.

Launch geometry caused the first apparent regression. A binary compiled for 152 persistent workers ran on a 148-SM B300 and left four CTAs in an almost empty second wave. Recompiling for 148

**Table 11:** Hardware properties relevant to the measured kernels. Per-SM exponential rates are reported by FA4; GPU-level Ultra ratios are from NVIDIA.

| Property | GB200 / SM100 | B300 / SM103 |
|---|---|---|
| Visible SMs in this study | 152 | 148 |
| Maximum reported clock | 2062 MHz | 2032 MHz |
| TMEM per SM | 256 KB | 256 KB |
| Key exponential rate | 16 ops/clock/SM | 32 ops/clock/SM |
| Dense NVFP4 GPU class | 1.0× | 1.5× |
| Fused TMEM load + reduction | no | `tcgen05.ld.red` |

removes that tail. The retained D128 SM103 policy then uses six K/V stages, fused score reduction, and only a light quarter-3 EX2 route where it wins. Routing EX2 through every quarter ties or loses despite the stronger SFU. The full density and grid sweep is moved to Appendix B.1.

At S8192/H64 and S9472/H64, enough work is available to sustain 3116 and 3159 TFLOP/s. S9472 has 2368 logical query-pair jobs, exactly 16 per persistent CTA. At S32768/H24, the tested B300 instead reaches 2945 TFLOP/s versus 2998 on GB200. Its visible SM-clock envelope is 4.05% smaller; normalized per SM-clock, B300 is 2.35% faster. The normalization explains the direction but is not an observed throughput result.

## 7.4 D64 specialization

HAO's published D64 full-FP4 route hits an atom boundary: its scaled FP4 PV atom requires K128. Our D64 path uses dedicated K64 NVFP4/MXFP4 atoms. On SM103 it also uses `tcgen05.ld.red` for all four score quarters and sends eight of each quarter's 16 score pairs through native EX2.

Across all 24 matched shapes, safe NV/MX on B300 is 1.042× faster than GB200 geometrically, and 1.058× for $S \geq 4096$. At S32768/H24 it reaches 1434 TFLOP/s in 4.601808 ms. A raw NV/NV control is faster on some inputs but produces non-finite values at H32/S32768 for one seed; the bounded control and full matrix are in Appendix A.6.

## 7.5 What would move the boundary

The remaining useful changes require a different hardware or ownership contract:

**Scaled-FP4 K32 PV.** Consume each N32 completion immediately instead of waiting for a pair.

**Another score bank.** Give QK a destination while PV still owns the current P overlay.

**Scales outside TMEM.** Let the matrix instruction consume compact scales directly from shared memory or registers.

**Compressed score ownership.** Create look-ahead without duplicating a full 128-column FP32 score tile.

**Wider SM103 tiles.** K96/K64 Ultra instructions make N192 or N256 attractive, but only if two-query K/V reuse and the P lifecycle survive the rewrite.

# 8 Discussion and Limitations

## 8.1 What the retained route establishes

Stabilized NV/NV remains the higher-fidelity full-FP4 control. Its row reference, E4M3 scale construction, and inverse correction are expensive on the online path; removing them makes NV/NV distribution-dependent. NV/MX chooses a different compromise. Its E8M0 scale represents small N32 probability blocks without a tensor-wide P factor and folds into the direct log-domain code map, at the cost of coarser scale placement.

The resulting two-times-class advantage over BF16 is a kernel result, not the four-times peak matrix ratio. BF16 FA4 already overlaps much of its non-matrix work. FP4 shortens QK and PV but still loads FP32 scores, waits for K64-ready pairs, publishes scales, updates online state, and stores output. The fixed-P ceiling shows that the remaining P arithmetic alone is no longer enough to close that gap.

The attribution is equally important. The two-query CTA, full 512-column TMEM layout, warp specialization, deep K/V ring, and split P publication come from HAO. This work contributes the mixed NV/MX representation, direct E2M1 code map, SFU/FMA balance, represented denominator, sampled range guard, and its underflow-safe evaluation.

## 8.2 Numerical evidence and its limits

Operator cosine and relative-$L_2$ are worse than HAO NV/FP8, while the tested ViT, BERT, and ViT-MAE tasks show much smaller final changes. Residual paths, normalization, later layers, and decision margins explain why these measures need not move together. They do not prove that the approximation is safe for every model or for training.

Wan provides a harder depth test. The corrected fast route is finite through 20 steps on calibration and held-out prompts, but its error grows more than HAO NV/NV and NV/FP8. Layer-wise coefficient fitting also fails to transfer reliably because a single-layer BF16-teacher proxy does not model interacting errors in an all-FP4 trajectory. Future calibration should optimize a small shared codebook under the end-task loss, ideally through QAT, and validate on fresh prompts and processes.

The present suite is forward-only and noncausal. It excludes dynamic Q/K/V quantization, folded-scale preparation, and K/V permutation cost. It does not establish backward stability, causal decoding, GQA, variable-length support, training convergence, language-model perplexity, long-context retrieval, or full video-quality metrics. SageAttention3 and Attn-QAT provide the relevant larger evaluation templates [11, 12].

## 8.3 Hardware dependence

This design is specific to SM100/SM103's TMEM and scaled-FP4 instruction contracts. B300 improves favorable saturated shapes, but it has the same 256-KB TMEM capacity and the tested SKU exposes fewer SMs at a slightly lower clock. One all-TMEM CTA still resides per SM, and broad use of its faster SFU does not remove the P-to-PV dependency. A GPU with another score bank, K32 scaled PV, or direct compact-scale operands could prefer a deeper pipeline. Conversely, SageAttention3's SM120 result informs the numerics but not this TMEM schedule.

# 9 Conclusion

Full-FP4 FlashAttention-4 is limited by the path that turns online scores into a legal FP4 probability operand, not by FP4 matrix instructions alone. Within HAO's two-query TMEM pipeline, NVFP4 QK plus MXFP4 P/V, direct E2M1 code mapping, represented normalization, and early K64 publication recover a two-times-class BF16 speedup. A sampled range guard and underflow-safe denominator keep the same path finite on 20-step Wan replays.

The result is a speed–accuracy frontier rather than strict dominance of HAO's higher-fidelity NV/FP8 route. Larger model and training evaluations are needed, while further throughput gains likely require K32 consumption, more TMEM buffering, or a different scale-delivery contract.

# References

[1] Kyle Aubrey and Nick Stam. Inside nvidia blackwell ultra: The chip powering the ai factory era. NVIDIA Technical Blog, August 2025. URL https://developer.nvidia.com/blog/inside-nvidia-blackwell-ultra-the-chip-powering-the-ai-factory-era/.

[2] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL https://arxiv.org/abs/2307.08691.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35, 2022. URL https://arxiv.org/abs/2205.14135.

[4] HAO AI Lab. Fp4 flash attention 4 on blackwell. GitHub repository, branch `fp4`, commit `9b0abef`, 2026. URL https://github.com/hao-ai-lab/flash-attention-fp4/tree/fp4/flash_attn/cute.

[5] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. URL https://arxiv.org/abs/2111.06377.

[6] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, 2014. URL https://arxiv.org/abs/1405.0312.

[7] NVIDIA Corporation. Parallel thread execution isa, version 8.8, 2026. URL https://docs.nvidia.com/cuda/parallel-thread-execution/.

[8] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL https://arxiv.org/abs/2407.08608.

[9] Wan Team. Wan: Open and advanced large-scale video generative models, 2025. URL https://arxiv.org/abs/2503.20314.

[10] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling, 2026. URL https://arxiv.org/abs/2603.05451.

[11] Jintao Zhang, Jia Wei, Haoxu Wang, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Kai Jiang, Jianfei Chen, and Jun Zhu. Sageattention3: Microscaling fp4 attention for inference and an exploration of 8-bit training, 2025. URL https://arxiv.org/abs/2505.11594.

[12] Peiyuan Zhang, Matthew Noto, Wenxuan Tan, Chengquan Jiang, Will Lin, Wei Zhou, and Hao Zhang. Attn-qat: 4-bit attention with quantization-aware training, 2026. URL https://arxiv.org/abs/2603.00040.

# A Historical NV/NV Investigation

This appendix preserves the NV/NV work that led to the retained design. It is intentionally separated from the principal result because those kernels use a different P scale format, approximation policy, and sometimes a different denominator. Their timings must not be mixed with the final NV/MX headroom.

## A.1 Initial stabilized path

The first numerically robust full-FP4 route followed the conventional order:

$$z \to m \to e^{z-m} \to a_B \to s_{\mathrm{E4M3}} \to \mathrm{E2M1}. \tag{29}$$

It computed an exact online row maximum, produced floating probabilities, reduced an N32 block maximum, encoded an E4M3 scale, divided by that scale, and packed E2M1. This path remained finite on downstream inputs but measured about 0.1884 ms at S4096/H24. The sequence was numerically conservative and serial: scale selection could not start until floating probabilities existed.

For a global NV encode factor $G$, payload and scale can be expressed as

$$S_B(G) = Q_{\mathrm{E4M3}}\left(G\frac{a_B}{6}\right), \tag{30}$$

$$q_j(G) = Q_{\mathrm{E2M1}}\left(\frac{Gp_j}{S_B(G)}\right), \tag{31}$$

$$\widehat{p}_j(G) = \frac{S_B(G)}{G} q_j(G). \tag{32}$$

If $G$ multiplies both numerator and denominator consistently and no rounding boundary changes, it cancels. It is not a free accuracy knob once scale rounding, underflow, and saturation are considered. Sweeping $G$ without changing the shiftless estimator produced little benefit; adding full stabilization to make the sweep meaningful restored the expensive path.

## A.2 Code-directed polynomial development

The key useful idea from NV/NV was to target E2M1 decisions directly. A cubic fit to the base-two locations of E2M1 thresholds was

$$\begin{aligned} p(x) = {} & 0.07839806x^3 + 0.28625049x^2 \\ & + 0.63145205x + 0.99202336. \end{aligned} \tag{33}$$

After folding the score and block scale into its coefficients, two values could be evaluated with packed Horner operations:

$$r_1 = \mathrm{FMA2}(x, a, b), \tag{34}$$
$$r_2 = \mathrm{FMA2}(r_1, x, c), \tag{35}$$
$$y = \mathrm{FMA2}(r_2, x, d). \tag{36}$$

This was faster than a full exponential and more accurate than early linear fits. It also established that the fitting target should be $Q_{\mathrm{E2M1}}(f(x))$, not $f(x)$ itself.

An intermediate SFU/ALU hybrid issued four native `EX2` pairs first and filled their latency with twelve cubic or affine pairs. The native samples also supplied an approximate denominator. This improved over pure-SFU and pure-ALU variants on that schedule, but it coupled numerator and denominator sampling to one distribution.

The later affine endpoint

$$\ell(x) = \max(0, 1.62330034x + 0.92083546) \tag{37}$$

reduced non-native work to one packed FMA per pair. That mechanism survives in the retained NV/MX path, with policy-specific coefficients and an exact represented denominator.

## A.3 Sampled denominator

The NV/NV throughput path sampled eight of 32 scalar probability values in each quarter. If $\mathcal{J}_q$ is the rotated sample set, it estimated

$$\widetilde{L}_B = A_B \, 4 \sum_{j \in \mathcal{J}_q} 2^{x_j}. \tag{38}$$

The factor four compensates for sampling one quarter of the values. This saved a complete denominator pass but could miss a concentrated maximum or misestimate a nonuniform tail. Two- and three-word denominator reductions were faster diagnostics but failed downstream: in a 20-image ViT gate they gave respectively 0% and 75% top-1 agreement with BF16. The retained NV/MX path instead sums all represented E2M1 payload words.

## A.4 Policy ladder and distribution dependence

The NV/NV investigation exposed `fast`, `universal`, and `hao-l2` style policies. `Fast` used the most aggressive shiftless approximation. `Universal` restored scale guards and a more reliable row reference. `Hao-l2` retained full stabilization. Their rough latency hierarchy at S4096/H24 was approximately 0.10, 0.11–0.13, and 0.188 ms, respectively, depending on the exact generation of the kernel.

The important result was not one latency number. Fast NV/NV could have good cosine on Gaussian scores and still generate non-finite or highly distorted model outputs when its row-reference assumptions changed. Stabilization fixed those cases but consumed the desired speedup. Fixed-scale sweeps on the shiftless estimator did not reproduce the accuracy of the fully stabilized E4M3 control.

## A.5 Why NV/NV was not retained

NV/NV contributed three ideas that remain: code-directed approximation, early partial P publication, and the need to evaluate speed and downstream accuracy together. It was superseded for four reasons:

1. an E4M3 P scale without a robust global factor can round low-amplitude N32 blocks to zero; applying the factor correctly fixes the format-level underflow but restores work on the exposed P path;
2. independent block-16 scale work does not align as naturally with the N32 software producer;
3. the robust stabilized path was too slow, while the fastest shiftless path was distribution-sensitive;
4. the HAO-derived 512-column layout and MXFP4 scale handoff provided a cleaner way to retain small P blocks without adding another score bank.

The fixed-schedule format table remains in the main paper because it is a useful control: NV/NV can outperform NV/MX on a specific synthetic metric. The format-range and downstream experiments explain why that isolated result is not sufficient to select the production route.

## A.6 B300 D64 sweep and range failure

The D64 specialization provides a wider test of this distinction. Its B300 NV/MX route combines fused TMEM load/reduction with eight native-EX2 pairs and eight affine pairs per score quarter. Every NV/MX output in the 24-shape matrix is finite. The raw shiftless NV/NV control is often locally more accurate because its E4M3 scale can fit a benign block more closely, but it is finite on only 23 of the 24 tested cases.

**Table 12:** Selected B300 D64 format diagnostics. NV/MX is the retained finite route. NV/NV timings are shown only to explain the rejected control and must not be treated as production performance when the status is non-finite.

| $H$ | $S$ | NV/MX | | | | raw NV/NV | | | Status |
|---|---|---|---|---|---|---|---|---|---|
| | | ms | TFLOP/s | Cosine | Rel.-$L_2$ | ms | Cosine | Rel.-$L_2$ | |
| 12 | 4096 | 0.066528 | 775 | 0.955818 | 0.295719 | 0.070464 | 0.962281 | 0.278304 | finite |
| 24 | 4096 | 0.093248 | 1105 | 0.955551 | 0.296653 | 0.099296 | 0.962063 | 0.279040 | finite |
| 24 | 32768 | 4.601808 | 1434 | 0.956089 | 0.295010 | 4.842496 | 0.962065 | 0.279155 | finite |
| 32 | 2048 | 0.039872 | 862 | 0.954553 | 0.299511 | 0.039968 | 0.961772 | 0.280428 | finite |
| 32 | 32768 | 6.115360 | 1438 | 0.957303 | 0.291325 | 6.465520 | – | – | non-finite |
| 64 | 1024 | 0.025568 | 672 | 0.952035 | 0.306387 | 0.025632 | 0.960558 | 0.286304 | finite |
| 64 | 4096 | 0.207840 | 1323 | 0.955838 | 0.295956 | 0.218176 | 0.962128 | 0.278683 | finite |
| 64 | 32768 | 12.223360 | 1439 | 0.957248 | 0.291501 | 12.919808 | 0.962800 | 0.276050 | finite |

Across S4096-and-longer rows, retained NV/MX is 1.124–1.284× faster than the matched B300 BF16 kernel. Density two wins or ties density one on 23 of 24 shapes. H32/S2048 and H64/S1024 need 136-CTA and 128-CTA grid caps, respectively, because their short logical job counts interact poorly with the full 148-worker persistent grid. Other shapes retain 148 CTAs. Every promoted build uses 128 registers, one barrier, and no local-memory spills.

The decisive failure occurs at H32/S32768. Seed 20260802 produces exactly 64 non-finite raw NV/NV outputs while BF16 and NV/MX remain finite; another seed passes. Saturating the E4M3 scale encoder repairs the failing distribution at 6.670336 ms and 1.186× BF16, with 0.962921 cosine and 0.275742 relative-$L_2$. This bounded mode costs roughly 2–3%. MXFP4's E8M0 scale is not finer than E4M3, but its exponent range represents tiny probability blocks without a separately materialized row shift. Raw NV/NV can therefore win a local error metric when it remains in range; NV/MX is the retained low-latency route because it remains finite across the matrix.

### A.7 Measured downstream failure mechanism

The expanded provider matrix isolates the failure. Native HAO and the TK fixed-schedule control receive the same NVFP4-quantized Q, K, and V tensors. HAO first subtracts a row maximum. The TK control instead forms a shiftless block scale from

$$s_B^{\text{shiftless}} = \frac{1}{6} \max_{j \in B} \exp(z_j), \tag{39}$$

then encodes it in E4M3. Model scores make this quantity exceed the finite E4M3 maximum of 448 in every tested workload. This control has no satfinite guard, so its out-of-range scale conversion contaminates the dependent P and denominator path and produces non-finite context rows.

With row-max stabilization,

$$s_B^{\text{stable}} = \frac{1}{6} \max_{j \in B} \exp(z_j - m), \qquad m = \max_j z_j, \tag{40}$$

so $0 \le s_B^{\text{stable}} \le 1/6$. Table 13 reports the first-sample diagnostic. The non-finite count is accumulated over all attention layers evaluated for that sample. Small stabilized blocks can still underflow, but they contain negligible probability mass relative to the row anchor; HAO remains finite on every full replay.

**Table 13:** Shiftless TK NV/NV failure on model activations. Overflow is the fraction of N32 P scales above E4M3's maximum before encoding.

| Task | Failed sample | Non-finite rows | Shiftless overflow (%) | Shiftless max | Stable overflow (%) | Stable max |
|---|---|---|---|---|---|---|
| ViT S256 | 1 | 2,560 | 8.146 | 1.828e+37 | 0.000 | 0.166667 |
| ViT S1024 | 1 | 127,950 | 8.137 | 5.332e+37 | 0.000 | 0.166667 |
| ViT S4096 | 1 | 539,442 | 7.511 | 5.669e+37 | 0.000 | 0.166667 |
| BERT MLM S256 | 1 | 167 | 1.721 | 5.279e+04 | 0.000 | 0.166667 |
| BERT MLM S512 | 1 | 49,613 | 1.063 | 5.655e+04 | 0.000 | 0.166667 |
| BERT SST-2 S256 | 1 | 21,717 | 1.465 | 4.493e+03 | 0.000 | 0.166667 |

## B Rejected Kernel Directions

### B.1 SM103 launch and EX2 controls

Table 14 records the initial B300 compatibility and native-EX2 density sweep. It is kept outside the main result table because it is a tuning control, not a set of promoted policies. The 152-worker binary creates a partial second wave on the tested 148-SM SKU; the corrected rows use 148 workers and vary EX2 density while holding the numerical contract fixed.

**Table 14:** Measured TK NV/MX tuning points on B300. Every row reports speed and error from the same output.

| Variant | Grid | EX2 density | Time (ms) | TFLOP/s | Cosine | Rel.-$L_2$ | RMSE |
|---|---|---|---|---|---|---|---|
| B300 compatibility | 152 | 0 | 0.123872 | 1664 | 0.943964 | 0.335825 | 0.008716 |
| B300 density 0 | 148 | 0 | 0.093216 | 2212 | 0.943964 | 0.335825 | 0.008716 |
| B300 density 1 | 148 | 1 | 0.097248 | 2120 | 0.947037 | 0.325258 | 0.008442 |
| B300 density 2 | 148 | 2 | 0.097280 | 2119 | 0.950069 | 0.314812 | 0.008171 |
| B300 density 4 | 148 | 4 | 0.111584 | 1848 | 0.956047 | 0.294037 | 0.007632 |

## B.2 Kernel directions

Table 15 condenses the major negative experiments. These results are retained because they constrain future work; compile-time probes remain default-off.

The raw cases for the retired intermediate policy remain in the unified suite for provenance, but it is excluded from the published CSV, plots, and policy tables.

## B.3 Quarter-local speed-of-light experiment

On the historical 0.102400 ms NV/NV path, replacing selected quarter transforms with raw packing measured:

| Transform removed | Time (ms) | Gain |
|---|---|---|
| Q0 only | 0.104736 | none |
| Q1 only | 0.102400 | none |
| Q0+Q1 | 0.099328 | 4.064 $\mu$s |
| Q2+Q3 | 0.096416 | 7.008 $\mu$s |
| all quarters | 0.090112 | 12.288 $\mu$s |

The pair behavior is the important finding: accelerating one N32 producer does not advance a K64 tensor command. The 12.288 $\mu$s total is not the current NV/MX ceiling; Section 7.1 reports the final 2.336 $\mu$s raw-score-pack gap.

## B.4 Scale-footprint experiments

Several probes attempted to reduce scale storage by sharing scales across 32×32 blocks, folding adjacent IDs, or moving source scales through shared memory. Compact source storage is useful, but `tcgen05` still consumes an instruction-facing scale layout. A smaller source tensor therefore does not automatically create free tensor-memory columns. The retained folded K64 Q/K representation reduces loads and arithmetic; it does not claim a new 128-column score slot.

## B.5 Joint affine-route search

We also optimized the affine E2M1 coefficients across all non-guard layers at once. Exact, equal-cost binaries were screened with persistent model workers; the optimizer combined favorable single-layer

**Table 15:** Major rejected directions. A timing tie is not promoted when it adds synchronization, storage, or numerical risk.

| Direction | Intended benefit | Observed failure mechanism |
|---|---|---|
| Half-tile QK/PV | Larger tensor work and easier overlap | Delayed first publication and increased live tensor-memory pressure; did not beat N32 production with K64 consumption. |
| Deeper dynamic scheduler | Exploit QK running one logical step ahead | Polls, proxy signals, and policy branches added control work without creating a legal score destination. |
| Full or QK-only two-CTA | Accelerate QK and increase occupancy | Cluster-wide readiness and scale lifetime coordination overwhelmed QK savings; QK-only did not remove single-CTA P/PV ownership. |
| Extra barriers or offload WG | Remove full-CTA rendezvous | Duplicate score loads and handoff mailboxes cost more than the hidden work; concurrent TMEM writes produced invalid output. |
| Alternate TMEM layouts | Add a second useful score/P slot | Two 128-column scores plus two 128-column outputs already consume 512 columns. Scale compression freed fragments, not another legal 128-column bank. |
| BF16/FP16 partial accumulator | Halve output columns | Local scaled-FP4 tensor instructions accumulate into FP32 TMEM; casting between issues did not change the accumulator contract. |

**Table 16:** Major rejected directions (continued).

| Direction | Intended benefit | Observed failure mechanism |
|---|---|---|
| Raw FP4 or coarse 2-D scales | Remove scale pages | Raw E2M1 loses the four-times-class block-scaled primitive. A single 32×32 scale cannot be applied after a reduction whose block product scales vary with K. |
| Direct code classifier | Eliminate packed conversion | Threshold trees, integer conversion, LUT access, LOP3, and PRMT packing generated more SASS than packed FFMA2 plus native F2FP. |
| Intermediate NV/MX policy | Add an anchor without the correction warpgroup | At S4096/H24 it measured 0.094560 ms, slower than `fast`, while its 0.356700 relative-$L_2$ was worse than both `fast` and `accurate`. Its long-ViT agreement only tied `accurate`, leaving no Pareto value. |
| Quadratic/cubic throughput path | Improve code fit | A Q0 quadratic raised static FFMA2 count from 128 to 160, measured 0.097888 ms, and reduced cosine on its test. |
| Sampled max/denominator | Shorten P preparation | Eight-of-32 samples saved less than 0.5 $\mu$s with substantial error; fewer samples were unstable. |
| Structured sparse PV | Increase tensor throughput | Blackwell's sparse FP4 path uses logical K128, losing the early K64 handoff; value-aware selection added too many instructions. |
| Tail interleaving | Pull Q3 work under Q2/PV latency | Added 1.4–3.2 $\mu$s; the contiguous Q2-then-Q3 schedule was locally better. |
| Streaming denominator | Hide exact denominator reduction | Preserved output exactly but slowed the clean fast build by 2.11% and 1.83%. |

changes and then sampled full categorical layer maps. This produced large apparent replay gains, but fresh model processes rejected every proposed production change.

For Wan14B, the best regularized map changed four layers and reduced reused- worker mean relative-$L_2$ from 0.398533 to 0.392793 over four prompts. On a fresh city run it instead increased relative-$L_2$ from 0.432932 to 0.453884. A fresh forest run also became non-finite in guarded layer 38, but that event was later traced to the E8M0 denominator-underflow bug described with the main Wan results; it is not evidence for or against the affine map. For Wan1.3B, a single layer-9 change reduced reused-worker mean relative-$L_2$ from 0.299884 to 0.290924. Fresh coastal and snow runs both regressed: 0.247742 to 0.258156 and 0.230262 to 0.233996, respectively.

The route constants do not change kernel latency; the rejection is purely numerical. Reused-pipeline ordering can move the final diffusion latent by more than the candidate margin and make a small ranking unstable. A statistically sound joint optimizer would therefore need to score each route across several fresh model processes and held-out prompts. That cost is not justified here because it cannot improve throughput and the observed numerical margins are below process-to-process variation. We keep the joint optimizer as a diagnostic and require fresh-process, unseen-prompt validation for any future calibration change. The production manifest retains only the global $(1.60, 0.95)$ pair. Full search and control artifacts are in `results/fp4_fa4_wan_joint_20260806`.

## C Accuracy-Matched FP8 Control

The principal NV/MX result is a speed–accuracy trade-off, so a separate control asks what happens when the TK route is required to match HAO's reported NV/FP8 cosine. Table 17 uses the B1/S32768/H24/D128 B300 record. The HAO rows are reconstructed from published TFLOP/s and are cross-run context [4]; HAO does not publish relative-$L_2$.

**Table 17:** Accuracy-matched B300 control. Superscript p marks HAO-published cross-run values.

| Provider | Time (ms) | TFLOP/s | Cosine | Relative-$L_2$ |
|---|---|---|---|---|
| TK NV/MX fast | 4.481 | 2945 | 0.9429 | 0.3389 |
| TK NV/FP8 optimized | 7.584 | 1740 | 0.9573 | 0.2913 |
| TK NV/FP8 exact | 8.854 | 1490 | 0.9897 | 0.1433 |
| HAO NV/FP8$^{p}$ | 4.929 | 2677 | 0.9899 | – |
| HAO BF16$^{p}$ | 8.607 | 1533 | 1.0000 | – |

The exact TK NV/FP8 route reaches 0.9897 cosine, effectively matching HAO's published 0.9899 at the displayed precision. It takes 8.854 ms and reaches 1490 TFLOP/s, compared with HAO's 4.929 ms and 2677 TFLOP/s. It also slightly trails HAO's published BF16 throughput. The intermediate optimized TK NV/FP8 route improves speed but reaches only 0.9573 cosine. This experiment separates two conclusions: the HAO-derived pipeline is structurally viable, while the large speed of our retained NV/MX mode comes from making the exposed P path cheaper and accepting lower operator fidelity. Matching HAO's accuracy with the current exact TK FP8 path gives back that advantage.

# D Complete Fixed-Schedule Format Matrix

The main text reports the headline fixed-schedule control because the retained-policy frontier is the principal result. Table 18 provides the corresponding control at every unified shape. The four TK rows at each shape use one transform and scheduling budget while changing only the QK and P/V scale formats. Native HAO NV/NV and BF16 are included as external controls. Timing and all three error metrics come from the same deterministic shape record and use the same canonical BF16 reference as the main tables.

**Table 18:** Complete fixed-schedule format matrix. Each row reports latency and output error together; no timing is paired with accuracy from another run.

| Shape | Provider | Time (ms) | Speedup | Cosine | Relative-$L_2$ | RMSE |
|---|---|---|---|---|---|---|
| B1/S256/H16 | TK NV/NV fixed schedule | 0.010560 | 1.165× | 0.952388 | 0.320599 | 0.032716 |
| | TK MX/NV fixed schedule | 0.010528 | 1.169× | 0.948314 | 0.320161 | 0.032672 |
| | TK NV/MX fixed schedule | 0.010880 | 1.131× | 0.929613 | 0.373055 | 0.038069 |
| | TK MX/MX fixed schedule | 0.010528 | 1.169× | 0.924689 | 0.380760 | 0.038856 |
| | HAO NV/NV | 0.014336 | 0.858× | 0.982173 | 0.188921 | 0.019279 |
| | HAO BF16 | 0.012304 | 1.000× | 1.000000 | 0.000000 | 0.000000 |
| B1/S1024/H24 | TK NV/NV fixed schedule | 0.018240 | 1.245× | 0.960499 | 0.285533 | 0.014735 |
| | TK MX/NV fixed schedule | 0.018176 | 1.249× | 0.953730 | 0.301114 | 0.015539 |
| | TK NV/MX fixed schedule | 0.018432 | 1.232× | 0.946337 | 0.323387 | 0.016688 |
| | TK MX/MX fixed schedule | 0.018432 | 1.232× | 0.937614 | 0.350956 | 0.018111 |
| | HAO NV/NV | 0.033152 | 0.685× | 0.981844 | 0.190610 | 0.009836 |
| | HAO BF16 | 0.022704 | 1.000× | 1.000000 | 0.000000 | 0.000000 |
| B1/S2048/H24 | TK NV/NV fixed schedule | 0.041248 | 1.492× | 0.961651 | 0.281034 | 0.010218 |
| | TK MX/NV fixed schedule | 0.041632 | 1.478× | 0.954486 | 0.298556 | 0.010855 |
| | TK NV/MX fixed schedule | 0.042080 | 1.463× | 0.948649 | 0.317135 | 0.011531 |
| | TK MX/MX fixed schedule | 0.041280 | 1.491× | 0.939444 | 0.347612 | 0.012639 |
| | HAO NV/NV | 0.072112 | 0.854× | 0.981680 | 0.191501 | 0.006963 |
| | HAO BF16 | 0.061552 | 1.000× | 1.000000 | 0.000000 | 0.000000 |
| B1/S4096/H24 | TK NV/NV fixed schedule | 0.104000 | 1.585× | 0.962721 | 0.276316 | 0.007167 |
| | TK MX/NV fixed schedule | 0.102688 | 1.605× | 0.955444 | 0.295344 | 0.007660 |
| | TK NV/MX fixed schedule | 0.104032 | 1.584× | 0.950622 | 0.311853 | 0.008088 |
| | TK MX/MX fixed schedule | 0.102976 | 1.600× | 0.941142 | 0.343985 | 0.008922 |
| | HAO NV/NV | 0.192512 | 0.856× | 0.981894 | 0.190474 | 0.004940 |
| | HAO BF16 | 0.164800 | 1.000× | 1.000000 | 0.000000 | 0.000000 |
| B1/S4096/H64 | TK NV/NV fixed schedule | 0.227328 | 1.631× | 0.962449 | 0.277441 | 0.007171 |
| | TK MX/NV fixed schedule | 0.235616 | 1.573× | 0.955133 | 0.296365 | 0.007660 |
| | TK NV/MX fixed schedule | 0.232128 | 1.597× | 0.950186 | 0.313010 | 0.008090 |
| | TK MX/MX fixed schedule | 0.234112 | 1.583× | 0.940680 | 0.345061 | 0.008918 |
| | HAO NV/NV | 0.438192 | 0.846× | 0.981743 | 0.191187 | 0.004941 |
| | HAO BF16 | 0.370688 | 1.000× | 1.000000 | 0.000000 | 0.000000 |
| B1/S8192/H64 | TK NV/NV fixed schedule | 0.861792 | 1.870× | 0.962801 | 0.276083 | 0.005045 |
| | TK MX/NV fixed schedule | 0.905248 | 1.780× | 0.955295 | 0.295814 | 0.005405 |
| | TK NV/MX fixed schedule | 0.888832 | 1.813× | 0.950828 | 0.311299 | 0.005688 |
| | TK MX/MX fixed schedule | 0.903488 | 1.784× | 0.941102 | 0.344364 | 0.006292 |
| | HAO NV/NV | 1.693696 | 0.951× | 0.981700 | 0.191467 | 0.003499 |
| | HAO BF16 | 1.611488 | 1.000× | 1.000000 | 0.000000 | 0.000000 |

# E Reproduction and Compile-Time Contract

### E.1 HAO-grid suite

Run from `tk_fa4/fp4_fa4_fwd`. The following command shows one representative shape and policy; repeat `-shape` for the rows in Eq. 26 and run `nvmx-fast` and `nvmx-accurate` in separate output directories.

```
CUDA_VISIBLE_DEVICES=0 python3 hao_comprehensive_suite.py \
  --shape 1,4096,24,128 \
  --variant nvmx-fast \
  --warmup-ms 300 --rep-ms 3000 \
  --cooldown-seconds 0.8 \
  --seed 20260814 --gpu 0 \
  --output-dir /path/to/fast_shard0 \
  --build-root /tmp/fp4_fa4_fast_shard0
```

The production replay used four identical GB200s. Each shard contains its own manifest; the generated summary validates exactly one complete case per shape and policy:

```
python3 \
  results/fp4_fa4_hao_table_gb200_20260802/build_summary.py
```

### E.2 FP8 and saturation controls

The independent wider suite adds FP8 PV and H64 shapes:

```
python3 hao_comprehensive_suite.py \
  --variant all \
  --shape 1,256,16,128 \
  --shape 1,1024,24,128 \
  --shape 1,2048,24,128 \
  --shape 1,4096,24,128 \
  --shape 1,4096,64,128 \
  --shape 1,8192,64,128 \
  --warmup-ms 300 --rep-ms 3000 \
  --seed 20260814 --gpu 0 \
  --output-dir /path/to/fp4_fa4_unified_replay
```

Its canonical BF16/HAO reference harness preallocates outputs and cycles through all six provider orders. Every order and per-window median remains in the JSON audit record.

### E.3 Downstream provider suite

The downstream matrix uses one adapter for TK and native HAO. Every provider receives identical weights, dataset order, seed, padding mask, and prepared Q/K/V values.

```
cd tk_fa4/fp4_fa4_fwd
python3 downstream_provider_suite.py \
  --gpu 0 \
  --output-dir \
  ../../results/fp4_fa4_downstream_matrix_20260801
```

Tasks and providers can be sharded with repeated `-task` and `-provider`. Regenerate the audit without rerunning models using `-summarize-only`. The suite hashes every full BF16 replay and checks a matching BF16 prefix for the fail-fast NV/NV control. HAO uses its native CuTe forward, not the TK port.

### E.4 Wan QK-guarded policy bundle

Wan uses a layer-routed bundle rather than one global fallback. The build script compiles the fast and accurate NV/MX bases, compiles the wide QK guard, and records the routing in one manifest. The following replays the 14B fast policy without exact stable softmax:

```
cd tk_fa4/fp4_fa4_fwd
python3 build_wan_nv_mx_bundle.py \
  --model 14b \
  --output-dir /tmp/wan_nv_mx_14b_s7680
CUDA_VISIBLE_DEVICES=0 \
PYTHONPATH=/workspace/codebases/flash-attention-fp4 \
python3 eval_wan_video.py \
  --model Wan-AI/Wan2.1-T2V-14B-Diffusers \
  --providers hao-bf16,tk \
  --policy-manifest \
    /tmp/wan_nv_mx_14b_s7680/manifest.json \
  --policy fast --steps 4 \
  --output /tmp/wan_14b_fast.json
```

Use `-policy accurate` for the represented-normalization base. The manifest selects guarded layers 27–29 for 1.3B and 33–34/38–39 for 14B; all other layers remain on the selected base policy. The default guard uses margin/stored-shift pairs 110/16 and 112/14, respectively. The builder rejects a combined shift above 126, and the kernel clamps translated E8M0 scales to code 1 before evaluating represented normalization in the non-underflowing order.

The main Wan table and its CuTe-BF16 paired outputs are in `results/fp4_fa4_wan_cute_bf16_20260806`. Rebuild the table with:

```
python3 results/fp4_fa4_wan_cute_bf16_20260806/\
build_tables.py
```

The corrected guard manifests, paired prompt replays, and five-second timing windows are in `results/fp4_fa4_wan_guard_fix_20260806`.

The layer-boundary calibration artifacts and aggregation script are in `results/fp4_fa4_wan_affine_calibration_20260806`. The route evaluator's `-active ROUTE=LAYERS` option runs FP4 only in selected layers and BF16 attention elsewhere, which keeps every candidate on the same teacher trajectory. Rebuild the audit table with:

```
python3 results/fp4_fa4_wan_affine_calibration_20260806/\
build_summary.py
```

The rejected end-to-end joint route search, wider prompt validation, and fresh-process controls are in `results/fp4_fa4_wan_joint_20260806`. The corresponding reusable tools are `joint_optimize_wan_affine.py`, `validate_wan_joint_routes.py`, and `materialize_wan_joint_route.py`.

## E.5 ViT-MAE reconstruction

Download the 100 recorded COCO 2017 validation filenames into one directory, then run each provider in a separate process so its extension and CuTe cache remain isolated. A representative fast run is:

```
cd tk_fa4/fp4_fa4_fwd
EXT=/tmp/tk_hao_unified_g0/
EXT=${EXT}b1_s256_h16_d128_nvmx-fast.so
CUDA_VISIBLE_DEVICES=0 python3 \
  eval_vit_mae_reconstruction.py \
  --image-dir /path/to/coco-val-images \
  --samples 100 \
  --extension $EXT \
  --output /tmp/nvmx_fast_100.json
```

For the accurate route, add `-global-anchor-kv`. For HAO controls, set `-attention-backend` to `hao-native` or `hao-fp8`. The input order is stored in every JSON file. Regenerate paired intervals and the table with:

```
python3 results/fp4_fa4_reconstruction_20260805/\
build_summary.py
```

## E.6 B300 mini-job

The committed compatibility job can be submitted with the Volt production CLI after exposing the existing `GITHUB_TOKEN` secret:

```
volt job submit \
  results/fp4_fa4_b300_tuning_20260802/volt_b300_smoke.yaml \
  --tag project:fp4-fa4 --tag purpose:report
```

The job clones fixed TK and HAO commits, builds for `sm_103a`, and stores hardware metadata, logs, the extension hash, and benchmark JSON under Volt artifacts.

The follow-up job corrects the persistent-grid width and compiles four native-EX2 densities from the same source and input tensors:

```
volt job submit \
  results/fp4_fa4_b300_tuning_20260802/\
volt_b300_density_sweep.yaml \
  --tag project:fp4-fa4 --tag purpose:report
```

The complete D64 runs and final no-override policy check use:

```
volt job submit results/fp4_fa4_b300_tuning_20260802/\
volt_b300_d64_nvmx_full_sweep.yaml
volt job submit results/fp4_fa4_b300_tuning_20260802/\
volt_b300_d64_nvnv_full_sweep.yaml
volt job submit results/fp4_fa4_b300_tuning_20260802/\
volt_b300_d64_promoted_policy.yaml
```

The corresponding successful job IDs are `f8ajtqmbvd8p`, `ig2tvyqoxtlz`, and `2bz33uyyvvo8`. The grid-cap sweep is `q6nb0jj73xik`; bounded NV/NV validation is `szpe9052wipe`. The final D128 cross-generation rows come from the order/KV-stage replay `ev0g6vopwujl`, saturated SFU/reduction sweep `y5yrv3k9dvxh`, S6144 prefetch/SFU sweep `oen3rbbipbtf`, and long-context job `xfby78ic4nql`. The 3-PFLOP/s discovery sweep is `ibwp4shzwfmi`; its independent five-window, two-seed confirmation is `d9e3yhudpu93`. Their specifications are `volt_b300_d128_3ktflops_sweep.yaml` and `volt_b300_d128_3ktflops_confirm.yaml`. After downloading the artifacts, regenerate the B300 JSON and LaTeX rows by running `python3 build_summary.py` from the `results/fp4_fa4_b300_tuning_20260802` directory.

### E.7 Required operand contract

Both retained policies require folded K64 Q/K scales selected by the MSE policy:

```
--nv-qk-fold-k64-scales both \
--nv-qk-fold-scale-select mse
```

`Accurate` also requires the same fixed 32-row permutation for K and V. Pairing a folded-scale binary with ordinary block-16 Q/K scales can produce plausible timing and invalid accuracy; the suite couples the binary and operand arguments.

### E.8 Retained compile-time policy

`Fast` uses a 12-stage K/V ring, all-affine mode 23, four-word represented denominators, paired scale reuse, wide three-input maxima, Q-scale preloading, early asynchronous P-scale handoff, and a 200/208/56 register split. `Accurate` uses 13 K/V stages, native EX2 samples, a 32-row anchor, and correction-warpgroup normalization.

Historical NV/NV, intermediate NV/MX, sparse, sampled-denominator, direct-code, quadratic, two-CTA, alternate-owner, and interleaving probes remain compile-gated and default-off.

## F Terminology

**Quarter** One N32 fragment read from an N128 score tile. Four producer quarters feed two K64 scaled-FP4 PV commands.

**Score bank** A 128-column FP32 TMEM region receiving QK and later hosting transient P/scale overlays.

**Output bank** A permanent 128-column FP32 PV accumulator for one query stage.

**Represented denominator** A normalizer summed from the E2M1 payload and decoded block scale actually consumed by PV.

**Early publication** Signaling a legal first K64 payload and scale pair before tail P construction completes.

**Speed-of-light diagnostic** An intentionally incorrect or semantically altered kernel that removes work to bound latency.

**Full FP4** Q, K, P, and V use E2M1 payloads with block scales. NVFP4 QK plus FP8 PV is a control, not full FP4.